

Índice

1	Entorno y Configuración (macOS / Linux)	1
1.1	Instalación de Utilidades GNU en Mac	1
1.2	Alias de Compilación (~/.zshrc)	1
1.3	Script de Verificación de Tipografía (hash.sh)	1
1.4	Configuración Básica de Editor (~/.vimrc)	1
1.5	Remapear Caps Lock de Forma Nativa en macOS	2
1.6	Validar Binario Activo	2
2	Teoría de Juegos Imparciales y Teorema de Sprague-Grundy	2
2.1	Conceptos Fundamentales de Posiciones	2
2.2	El Teorema de Sprague-Grundy y la función mex	2
2.3	Implementación Eficiente de Grundy Values con DP	2
3	Basics	2
3.1	Standard Competitive Programming Template	2
4	Math	3
4.1	Algebra	3
4.1.1	Hill Climbing Optimization 2D	3
4.1.2	Numerical Integration (Simpson's 1/3 Rule)	3
4.1.3	Polinomio	3
4.2	Combinatoria	4
4.2.1	Binomial Coefficients Modulo Prime	4
4.3	Geometry	4
4.3.1	Angle Radians	4
4.3.2	Angle Between Segments (Complex Coordinates)	4
4.3.3	Geometry Primitives (Complex Coordinates)	4
4.3.4	Convex Hull (Monotone Chain - Compact)	5
4.3.5	Lattice Points (Pick's Theorem)	5
4.3.6	Minimum Enclosing Circle (Welzl's Algorithm)	6
4.3.7	Integer Point	7
4.3.8	Point in Polygon (Ray Casting)	7
4.3.9	Point Location vs Line	8
4.3.10	Polygon Area (Shoelace Formula)	8
4.3.11	Segment Intersection (Anti-Overflow Version)	8
4.4	Teoría Juegos	8
4.4.1	XOR Basis (Linear Basis)	8
4.5	Teoría Numeros	9
4.5.1	BigInt Operations (Unsigned)	9
4.5.2	Prime Sieve	10

1. Entorno y Configuración (macOS / Linux)

1.1. Instalación de Utilidades GNU en Mac

Para utilizar la suite completa de GNU (como la cabecera universal `<bits/stdc++.h>`) e igualar con precisión el entorno de evaluación del concurso, instala la última versión de GCC y las utilidades de núcleo de GNU mediante Homebrew:

```
|| brew install gcc coreutils
```

1.2. Alias de Compilación (~/.zshrc)

Agrega este alias al final de tu archivo de configuración de la terminal para compilar localmente de forma rápida.

Nota Crítica de Arquitectura (Apple Silicon): En procesadores M1/M2/M3 bajo macOS, la suite `libsanitizer` dentro de GCC para ARM64 no cuenta con soporte nativo completo en Homebrew, lo que arroja el error `ld: library 'asan' not found`. Se han removido las banderas `-fsanitize=address,undefined`, pero se conservan los diagnósticos estrictos de tipos, desbordamientos lógicos y conversiones implícitas peligrosas:

```
|| # Ajustar g++-15 a la version exacta instalada en tu equipo (ej. g++-14, g++-15)
|| alias c='g++-15 -Wall -Wconversion -Wfatal-errors -g -std=c++17'
```

Actualiza tu sesión corriendo en la terminal: `source ~/.zshrc`.

También puedes simplemente agregarlo al final como: `echo .alias c='g++-15 -Wall -Wconversion -Wfatal-errors -g -std=c++17'>> ~/.zshrc`

1.3. Script de Verificación de Tipografía (hash.sh)

Crea el archivo `hash.sh` para validar la correcta transcripción de tus plantillas de código desde el cuaderno impreso. Este script compara el hash de 6 caracteres del código limpio con el hexadecimal de KACTL.

Nota sobre Portabilidad de Hashes: Es indispensable incluir la bandera `-fpreprocessed` para evitar que el compilador inyecte macros internos específicos del procesador M2 o del sistema operativo macOS, lo cual corrompería el hash portable de KACTL:

```
|| #!/bin/bash
|| g++-15 -E -dD -P -fpreprocessed - | tr -d '[:space:]' | gmd5sum | cut -c-6
```

Otorga los permisos de ejecución necesarios en tu terminal:

```
|| chmod +x hash.sh
```

Úsalo de la siguiente forma para verificar fallos tipográficos: `./hash.sh <codigo.cpp`

1.4. Configuración Básica de Editor (~/.vimrc)

Habilita el auto-indentado limpio para código de C++ y el mapeo de escape veloz presionando las teclas `jk` o `kj` de forma sucesiva en modo inserción:

```
|| set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul sy on
|| im jk <esc>
|| im kj <esc>
```

1.5. Remapear Caps Lock de Forma Nativa en macOS

Para evitar comandos de Linux incompatibles en macOS (como `xmodmap`), puedes cambiar el mapeo directamente desde la interfaz del sistema operativo:

1. Ve a **Ajustes del Sistema** → **Teclado**.
2. Haz clic en **Atajos de teclado...** y luego selecciona **Teclas de modificación**.
3. Cambia la acción de la tecla **Bloqueo de mayúsculas** (*Caps Lock*) a **Control** o **Escape** para una navegación ultra veloz en Vim.

1.6. Validar Binario Activo

Si Homebrew actualiza o instala una versión superior de GCC, identifica con exactitud el sufijo del comando y las rutas ejecutando:

```
brew info gcc
ls /opt/homebrew/bin/g++-
```

2. Teoría de Juegos Imparciales y Teorema de Sprague-Grundy

2.1. Conceptos Fundamentales de Posiciones

En la convención de juego normal (el último en mover gana), definimos de forma recursiva los estados del juego:

- **Estados Terminales:** Son posiciones P (puesto que el jugador actual no puede realizar movimientos y ha perdido).
- **Posición N (Next):** Es cualquier estado desde el cual existe **al menos un movimiento** hacia una posición P . (El jugador actual se mueve a P y le hereda la muerte al rival).
- **Posición P (Previous):** Es un estado desde el cual **todos los movimientos posibles** conducen obligatoriamente a posiciones N . (Hagas lo que hagas, dejas al rival en un estado ganador).

2.2. El Teorema de Sprague-Grundy y la función mex

El teorema dicta que cualquier juego imparcial bajo juego normal es equivalente a una pila de Nim. El tamaño de dicha pila ficticia para un estado G es su valor de Grundy, denotado como $g(G)$, y se calcula mediante la función del mínimo excluido (mex):

$$g(G) = \text{mex}(\{g(V) : G \rightarrow V\})$$

Donde $G \rightarrow V$ representa un movimiento válido desde el estado G hacia el estado V . El mex de un conjunto de enteros no negativos es el entero no negativo más pequeño que **no** pertenece a dicho conjunto (ej. $\text{mex}(\{0, 1, 3\}) = 2$, $\text{mex}(\{1, 2\}) = 0$).

2.3. Implementación Eficiente de Grundy Values con DP

Para calcular los valores de Grundy en juegos abstractos (como juegos de sustracción de fichas o movimientos en grafos acíclicos dirigidos - DAGs), mapeamos los estados usando Programación Dinámica y calculamos el mex optimizado con vectores booleanos locales o conjuntos:

```
// Calcula el Grundy Value para un conjunto de estados sucesores directos
int getMex(const vector<int>& transitions) {
    vector<bool> present(transitions.size() + 1, false);
    for (int val : transitions) {
        if (val < (int)present.size()) {
            present[val] = true;
        }
    }
    int mex = 0;
    while (present[mex]) mex++;
    return mex;
}

// Ejemplo de DP para un juego de sustraccion de fichas
// Se pueden retirar {1, 3, 4} fichas por turno.
int computeGrundy(int n) {
    vector<int> memo(n + 1, 0);
    vector<int> moves = {1, 3, 4}; // Movimientos permitidos

    for (int i = 1; i <= n; ++i) {
        vector<int> next_states;
        for (int m : moves) {
            if (i - m >= 0) {
                next_states.push_back(memo[i - m]);
            }
        }
        memo[i] = getMex(next_states);
    }
    return memo[n];
}
```

Estrategia ICPC: Si el valor devuelto por `computeGrundy(N)` es 0, el estado actual es una P -position (pierdes). Si es > 0 , es una N -position (ganas). Si el juego consta de múltiples sub-juegos independientes concurrentes, la respuesta global es el XOR de los valores individuales de Grundy de cada sub-juego: $G_{total} = g(G_1) \oplus g(G_2) \oplus \dots \oplus g(G_k)$.

3. Basics

3.1. Standard Competitive Programming Template

Title: Standard Competitive Programming Template

Description: Plantilla base con optimización de I/O, macros comunes y typedefs esenciales para reducir tiempo de codificación.

Author: Pumas Sangrientos

```
#include <bits/stdc++.h>

using namespace std;

// === Typedefs ===
typedef long long ll;
typedef long double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vll;
```

```

12 // === Macros ===
13 #define endl '\n'
14 #define pb push_back
15 #define all(x) (x).begin(), (x).end()
16 #define sz(x) (int)(x).size()
17 #define FOR(i, a, b) for(int i = (a); i < (b); ++i)
18 #define ROF(i, a, b) for(int i = (a); i >= (b); --i)
19
20
21
22 void fast_io() {
23     ios_base::sync_with_stdio(false);
24     cin.tie(NULL);
25     cout.tie(NULL);
26 }
27
28 void solve() {
29     // Escribe tu lógica aquí
30 }
31
32 int main() {
33     fast_io();
34
35     int t = 1;
36     // cin >> t; // Descomentar si el problema tiene múltiples casos de prueba
37     while(t--) {
38         solve();
39     }
40
41     return 0;
42 }

```

4. Math

4.1. Algebra

4.1.1. Hill Climbing Optimization 2D

Title: Hill Climbing Optimization 2D

Author: KACTL / botellot / ICPC BRO

Description: Heuristic search to minimize a 2D continuous function $f(x, y)$.

It explores a 3x3 local grid around the current state with an exponentially decaying jump size.

Complexity: $O(\log(\text{scale})) \cdot 100 \cdot 9 \approx O(1)$ with a small constant.

Limitations:

- Strictly requires the function to be unimodal (or locally well-behaved near 'start').
- Can get trapped in local minima if the surface has multiple valleys.

Usage:

auto [val, pos] = hillClimb({0, 0}, [](P p) { return p[0]*p[0] + p[1]*p[1]; });

```

1 // Macro utilitaria de KACTL
2 template<class F>
3 pair<double, P> hillClimb(P start, F f) {
4     pair<double, P> cur(f(start), start);
5     for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
6         rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2) {
7             P p = cur.second;
8             p[0] += dx * jmp;
9             p[1] += dy * jmp;
10            cur = min(cur, make_pair(f(p), p));
11        }
12    }
13    return cur;
14 }

```

4.1.2. Numerical Integration (Simpson's 1/3 Rule)

Title: Numerical Integration (Simpson's 1/3 Rule)

Author: KACTL / botellot / ICPC BRO

Description: Approximates the definite integral $\int_a^b f(x)dx$ (area under the curve) using Simpson's 1/3 rule by fitting parabolas over $2n$ subintervals.

Complexity: $O(N)$ where N is the number of intervals.

Limitations:

- The function 'f' must be continuous and smooth within $[a, b]$.
- If the function has sharp spikes, discontinuities, or vertical asymptotes, standard Simpson's rule will lose stability. In those cases, use Adaptive Integration instead.

Usage:

double area = quad(0, M_PI, [](double x) { return sin(x); }); // Returns 2.0

```

1 // Algoritmo de Integracion de KACTL
2 template<class F>
3 double quad(double a, double b, F f, const int n = 1000) { //
4     double h = (b - a) / 2 / n, v = f(a) + f(b); //
5     rep(i, 1, n * 2) //
6         v += f(a + i * h) * (i & 1 ? 4 : 2); //
7     return v * h / 3; //
8 }

```

4.1.3. Polinomio

Title: Polinomio

Author: KACTL / botellot / ICPC BRO

Description: Structure to store and manipulate real-coefficient polynomials.

Provides linear-time evaluation using Horner's Method, analytical differentiation, and in-place synthetic division (Ruffini's Rule) by a known root.

Complexity:

- Evaluation: $O(N)$
- Differentiation: $O(N)$
- Root Division: $O(N)$

Limitations:

- Uses 'double', so it is susceptible to floating-point precision loss and catastrophic cancellation in large degrees.
- 'divroot' assumes x_0 is an exact or highly precise root; otherwise, numerical errors accumulate in the lower coefficients.

Usage:

Poly p{{2, -3, 1}}; // Represents $x^2 - 3x + 2$

double val = p(2); // Evaluates $P(2) \rightarrow 0$

p.diff(); // $P(x)$ becomes $2x - 3$

```

1 // Macro utilitaria para emular el estilo scannable de KACTL
2 struct Poly {
3     vector<double> a;
4     // Evalúa el polinomio en el punto x usando el Método de Horner
5     // Evita calcular potencias explícitas x^n, reduciendo el error numérico.
6     double operator()(double x) const {
7         double val = 0;
8         for (int i = sz(a); i--;) {
9             val = val * x + a[i];
10        }
11        return val;
12    }
13    // Calcula la derivada analítica del polinomio in-place
14    // Regla de la potencia: d/dx(c * x^i) = i * c * x^(i-1)
15    void diff() {
16        for (int i = 1; i < sz(a); ++i) {
17            a[i - 1] = i * a[i];

```

```

18     }
19     if (!a.empty()) {
20         a.pop_back(); // El grado máximo disminuye en 1
21     }
22 }
23 // Divide el polinomio entre (x - x0) in-place usando la Regla de Ruffini
24 // Útil para "eliminar" una raíz ya encontrada y buscar las demás en un grado
25 // menor.
26 void divroot(double x0) {
27     double b = 0;
28     for (int i = sz(a); i--;) {
29         double c = a[i];
30         a[i] = b; // Guarda el coeficiente del nuevo cociente
31         b = c + b * x0; // Calcula el residuo acumulado
32     }
33     if (!a.empty()) {
34         a.pop_back(); // Elimina el último elemento sobrante (grado se reduce
35 // en 1)
36 }
37 }
38 };

```

4.2. Combinatoria

4.2.1. Binomial Coefficients Modulo Prime

Description: Calcula coeficientes binomiales (nCr) usando factoriales precomputados e inversos modulares. Requiere que m sea primo.

Author: botellot

Title: Binomial Coefficients Modulo Prime

```

1 const int MAXN = 1000005; // Ajustar según el problema
2 // 0 998244353
3 ll factorial[MAXN];
4 ll inv_factorial[MAXN];
5 // Exponenciación binaria para el inverso modular O(log m)
6 ll power(ll base, ll exp) {
7     ll res = 1;
8     base %= m;
9     while (exp > 0) {
10         if (exp % 2 == 1) res = (res * base) % m;
11         base = (base * base) % m;
12         exp /= 2;
13     }
14     return res;
15 }
16 ll modInverse(ll n) {
17     return power(n, m - 2);
18 }
19 // Precomputo en O(MAXN)
20 void precompute() {
21     factorial[0] = 1;
22     for (int i = 1; i < MAXN; i++)
23         factorial[i] = (factorial[i - 1] * i) % m;
24     // Inverso del factorial final usando Fermat
25     inv_factorial[MAXN - 1] = modInverse(factorial[MAXN - 1]);
26     // Inversos de los demás factoriales en O(MAXN)
27     for (int i = MAXN - 2; i >= 0; i--)
28         inv_factorial[i] = (inv_factorial[i + 1] * (i + 1)) % m;
29 }
30 // Cálculo en O(1)
31 ll nCr(int n, int k) {
32     if (k < 0 || k > n) return 0;
33     return factorial[n] * inv_factorial[k] % m * inv_factorial[n - k] % m;
34 }

```

4.3. Geometry

4.3.1. Angle Radians

Title: Angle <-> radians

```

1 const double PI = acos(-1.0);
2 // Convierte radianes a grados
3 double rad_to_deg(double rad) {
4     return rad * 180.0 / PI;
5 }
6 // Convierte grados a radianes (por si la necesitas al revés)
7 double deg_to_rad(double deg) {
8     return deg * PI / 180.0;
9 }

```

4.3.2. Angle Between Segments (Complex Coordinates)

Title: Angle Between Segments (Complex Coordinates)

Author: CP-Algorithms / ICPC BRO

Description: Computes the directed angle from segment AB to segment CD.

Returns the result in both radians and degrees.

Complexity: O(1).

```

1 using ftype = double;
2 using point = complex<ftype>;
3 const double PI = acos(-1.0);
4 // Retorna el ángulo dirigido desde v1 hacia v2 en radianes (rango de -PI a PI)
5 // Un valor positivo significa giro antihorario, negativo significa giro horario
6 double angle_between_vectors(point v1, point v2) {
7     // La magia de los complejos: arg(conj(v1) * v2)
8     // es equivalente a hacer atan2(cross(v1,v2), dot(v1,v2))
9     return arg(conj(v1) * v2);
10 }
11 // Retorna el ángulo dirigido del segmento AB al segmento CD
12 double angle_between_segments(point a, point b, point c, point d) {
13     point v1 = b - a; // Vector del primer segmento
14     point v2 = d - c; // Vector del segundo segmento
15     return angle_between_vectors(v1, v2);
16 }
17 void solve() {
18     // Segmento 1: De (0,0) a (2,0) -> Vector horizontal a la derecha (0 grados)
19     point a = {0.0, 0.0}, b = {2.0, 0.0};
20     // Segmento 2: De (0,0) a (0,2) -> Vector vertical hacia arriba (90 grados)
21     point c = {0.0, 0.0}, d = {0.0, 2.0};
22     double rad = angle_between_segments(a, b, c, d);
23     double deg = rad * 180.0 / PI;
24     cout << "Angulo en Radianes: " << rad << "\n"; // Debería dar PI / 2 (~1.5708)
25     cout << "Angulo en Grados: " << deg << "\n"; // Debería dar 90.0
26 }

```

4.3.3. Geometry Primitives (Complex Coordinates)

Title: Geometry Primitives (Complex Coordinates)

Author: CP-Algorithms / ICPC BRO

Description: 2D Geometry primitives using std::complex.

Maps (X, Y) to the complex number X + iY.

Provides native vector operations, dot product, cross product, and rotations.

Complexity: O(1) for all primitive operations.

```

// Podemos usar double o int64_t dependiendo de si el problema requiere precision
// exacta
using ftype = double;
using point = complex<ftype>;
// --- OPERACIONES BASICAS NATIVAS ---
// punto p = {x, y};
// p.real() -> Accede a la coordenada X
// p.imag() -> Accede a la coordenada Y
// p1 + p2 -> Suma de vectores (Traslacion)
// p1 - p2 -> Resta de vectores (Vector de p2 apuntando a p1)
// p * k -> Multiplicacion por un escalar k (Escalaiento)
// Producto Punto (Dot Product): a . b = |a||b|cos(theta)
// Utilidad: Proyecciones, detectar angulos agudos/rectos/obtusos
ftype dot(point a, point b) {
    return (conj(a) * b).real();
}
// Producto Cruz (Cross Product): a x b = |a||b|sin(theta)
// Utilidad: Areas, orientacion (CCW), giros izquierda/derecha
ftype cross(point a, point b) {
    return (conj(a) * b).imag();
}
// Magnitud al cuadrado (Squared Magnitude)
// Si ftype es entero, esto se mantiene 100% exacto sin floats
ftype norm_sq(point a) {
    return norm(a); // std::norm regresa x^2 + y^2
}
// Magnitud real (Distancia al origen)
double magnitud(point a) {
    return abs(a); // std::abs regresa sqrt(x^2 + y^2)
}
// Distancia entre dos puntos
double dist(point a, point b) {
    return abs(a - b);
}
// --- ORIENTACION Y GIROS ---
// Orientacion de 3 puntos (CCW)
// Retorna: > 0 si c gira a la izquierda de ab, < 0 a la derecha, 0 si son
// colineales
ftype ccw(point a, point b, point c) {
    return cross(b - a, c - a);
}
// --- ROTACIONES (Solo si ftype es punto flotante) ---
// Angulo polar del vector en radianes (rango de -pi a pi)
double angle(point a) {
    return arg(a);
}
// Rotar un punto 'a' un angulo 'theta' (en radianes) respecto al origen
point rotate_origin(point a, double theta) {
    return a * polar(1.0, theta);
}
// Rotar un punto 'p' respecto a un centro 'pivot' un angulo 'theta'
point rotate_pivot(point p, point pivot, double theta) {
    return pivot + (p - pivot) * polar(1.0, theta);
}
void solve() {
    point a = {3.0, 4.0};
    point b = {1.0, 2.0};
    cout << "X: " << a.real() << ", Y: " << a.imag() << "\n";
    cout << "Suma: " << (a + b).real() << " " << (a + b).imag() << "\n";
    cout << "Producto Punto: " << dot(a, b) << "\n";
    cout << "Producto Cruz: " << cross(a, b) << "\n";
}

```

4.3.4. Convex Hull (Monotone Chain - Compact)

Title: Convex Hull (Monotone Chain - Compact)

Author: botellot / ICPC BRO

Description: Computes the Convex Hull of a set of points using Andrew's Monotone Chain. Extremely compact version using a double-pass loop and coordinate reversing.

Complexity: $O(N \log N)$ due to sorting.

Usage:

vector<P> pts = {{0,0}, {3,0}, {3,3}, {0,3}, {1,1}};

vector<P> hull = get_convex_hull(pts);

```

struct P {
    ll x, y;
    void read() {
        cin >> x >> y;
    }
    P operator - (const P& b) const { return {x - b.x, y - b.y}; }
    ll operator * (const P& b) const { return x * b.y - y * b.x; }
    __int128 triangle(const P& b, const P& c) const {return (__int128)((b - *this)
        * (c - *this));}
    bool operator < (const P& b) const {return make_pair(x,y) < make_pair(b.x, b.y)
        ;}
};
// Funcion que recibe el set de puntos y retorna las esquinas del Convex Hull
vector<P> get_convex_hull(vector<P> points) {
    int n = points.size();
    if (n <= 1) return points;
    // 1. Ordenar de izquierda a derecha
    sort(points.begin(), points.end());
    vector<P> hull;
    // 2. Doble pasada (rep=0 para el cascaron superior, rep=1 para el inferior)
    for (int rep = 0; rep < 2; ++rep) {
        const int S = hull.size(); // S guarda el tamaño inicial de esta pasada
        for (P C : points) {
            // hull.size() - S >= 2 asegura no borrar puntos de la pasada anterior
            while ((int)hull.size() - S >= 2) {
                P A = hull[hull.size() - 2]; // Equivalente a hull.end()[-2]
                P B = hull[hull.size() - 1]; // Equivalente a hull.end()[-1]
                // Si el giro es a la derecha (o colineal), el punto B es valido
                if (A.triangle(B, C) <= 0) {
                    break;
                }
                // Si el giro es a la izquierda, B hace una "panza" invalida y se
                // elimina
                hull.pop_back();
            }
            hull.push_back(C);
        }
        hull.pop_back(); // Elimina el punto extremo duplicado antes de dar la
        // vuelta
        reverse(points.begin(), points.end()); // Voltear los puntos para el viaje
        // de regreso
    }
    return hull;
}

```

4.3.5. Lattice Points (Pick's Theorem)

Title: Lattice Points (Pick's Theorem)

Author: botellot / ICPC BRO

Description: Computes the number of interior (I) and boundary (B) lattice points of a simple polygon with integer coordinates using Pick's Theorem ($A = I + B/2 - 1$). Complexity: $O(N \log(\max_coord))$ due to GCD for each edge.

Usage:

vector<P> poly = {{0,0}, {0,3}, {4,0}};

auto [interior, boundary] = get_lattice_points(poly); // Returns {0, 12}

```

// Contiene std::gcd para C++17
struct P {
    ll x, y;
    void read() { cin >> x >> y; }
    P operator - (const P& b) const { return {x - b.x, y - b.y}; }
}

```



```

6 // __int128 para evitar overflow en coordenadas grandes durante el producto
7 cruz
8 __int128 operator * (const P& b) const { return (__int128)x * b.y - (__int128)y
9 * b.x; }
10 };
11 // Funcion iterativa de GCD para asegurar compatibilidad absoluta y velocidad
12 ll safe_gcd(ll a, ll b) {
13     a = abs(a); b = abs(b);
14     while (b) {
15         a %= b;
16         swap(a, b);
17     }
18     return a;
19 }
20 // Retorna un pair: {puntos_interiores, puntos_en_el_borde}
21 pair<ll, ll> get_lattice_points(const vector<P>& poly) {
22     int n = poly.size();
23     __int128 double_area = 0;
24     ll boundary = 0;
25     for (int i = 0; i < n; i++) {
26         int j = (i + 1 == n) ? 0 : i + 1;
27         // 1. Acumular el doble del area (Shoelace)
28         double_area += poly[i].x * poly[j].y;
29         // 2. Contar puntos enteros en el segmento actual usando GCD
30         P diff = poly[j] - poly[i];
31         boundary += safe_gcd(diff.x, diff.y);
32     }
33     // Asegurar que el area sea positiva
34     if (double_area < 0) double_area = -double_area;
35     // 3. Aplicar Teorema de Pick despejando los puntos interiores (I)
36     // Formula original: A = I + B/2 - 1 ==> 2A = 2I + B - 2 ==> 2I = 2A - B +
37     // 2
38     ll interior = (ll)(double_area - boundary + 2) / 2;
39     return {interior, boundary};
40 }
41 void solve() {
42     int n;
43     if (!(cin >> n)) return;
44     vector<P> polygon(n);
45     for (int i = 0; i < n; i++) {
46         polygon[i].read();
47     }
48     pair<ll, ll> ans = get_lattice_points(polygon);
49     cout << ans.first << " " << ans.second << "\n";
50 }
51 return 0;
52 }

```

4.3.6. Minimum Enclosing Circle (Welzl's Algorithm)

Title: Minimum Enclosing Circle (Welzl's Algorithm)

Author: CP-Algorithms / ICPC BRO

Description: Computes the Minimum Enclosing Circle (MEC) in expected $O(N)$ time.

Uses `std::variant` and `std::complex` for zero-allocation performance and clean logic.

Necesita `c++17` para `variant`, `visit` y `chrono`

Complexity: Expected $O(N)$.

Limitations: Coordinates must fit within `int64_t` operations inside `indicator()`.

```

1 using ftype = int64_t;
2 using point = complex<ftype>;
3 // El circulo minimo siempre esta representado por 2 o 3 puntos en su
4   circunferencia
5 using mec = variant<
6     array<point, 2>,
7     array<point, 3>
8 >;
9 ftype indicator(mec const& C, point z) {

```

```

9     return visit([&](auto &&C) {
10         point a = C[0], b = C[1];
11         // La parte real de Multiplicar un numero complejo por el conjugado de otro
12         (conj)
13         // es exactamente igual al producto punto de ambos vectores.
14         point IO = (b - z) * conj(a - z);
15         if constexpr (C.size() == 2) {
16             return real(IO);
17         } else {
18             point c = C[2];
19             point I2 = (a - c) * conj(b - c);
20             point I1 = IO * I2;
21             return imag(I2) < 0 ? -imag(I1) : imag(I1);
22         }
23     }, C);
24 bool inside(mec const& C, point p) {
25     return indicator(C, p) <= 0;
26 }
27 // Inicializacion del generador con un seed basado en tiempo real para evitar hacks
28 mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().count());
29 mec enclosing_circle(vector<point> &p) {
30     int n = p.size();
31     if (n == 0) return array<point, 2>{point{0, 0}, point{0, 0}};
32     // El shuffle aleatorio es lo que garantiza el tiempo esperado  $O(N)$ 
33     shuffle(p.begin(), p.end(), gen);
34     // Inicializar de forma segura con el primer punto duplicado (Soporta  $N=1$ 
35     // perfectamente)
36     mec C = array<point, 2>{p[0], p[0]};
37     for (int i = 0; i < n; i++) {
38         if (!inside(C, p[i])) {
39             C = array<point, 2>{p[i], p[0]};
40             for (int j = 0; j < i; j++) {
41                 if (!inside(C, p[j])) {
42                     C = array<point, 2>{p[i], p[j]};
43                     for (int k = 0; k < j; k++) {
44                         if (!inside(C, p[k])) {
45                             C = array<point, 3>{p[i], p[j], p[k]};
46                         }
47                     }
48                 }
49             }
50         }
51     }
52     return C;
53 }
54 // --- BONUS: Como recuperar el centro y el radio en punto flotante si el problema
55 // lo pide ---
56 pair<complex<double>, double> get_circle_geometry(mec const& C) {
57     return visit([&](auto &&C) -> pair<complex<double>, double> {
58         if constexpr (C.size() == 2) {
59             complex<double> a(real(C[0]), imag(C[0]));
60             complex<double> b(real(C[1]), imag(C[1]));
61             complex<double> center = (a + b) / 2.0;
62             double radius = abs(a - b) / 2.0;
63             return {center, radius};
64         } else {
65             complex<double> a(real(C[0]), imag(C[0]));
66             complex<double> b(real(C[1]), imag(C[1]));
67             complex<double> c(real(C[2]), imag(C[2]));
68             // Formula del circuncentro usando numeros complejos
69             complex<double> d = (a - b) * conj(c - b);
70             complex<double> center = b + (a - b) * complex<double>(0, 1) * (c - b)
71                 * conj(c - a) / (2.0 * d.imag());
72             double radius = abs(a - center);
73             return {center, radius};
74         }
75     }, C);
76 }
77 void solve() {
78     int n;

```

```

76 if (!(cin >> n)) return;
77 vector<point> orig_points(n);
78 for (int i = 0; i < n; i++) {
79     ftype x, y;
80     cin >> x >> y;
81     orig_points[i] = point{x, y};
82 }
83 // Sacamos una copia para que Welzl la barajee sin perder el orden original
84 vector<point> working_points = orig_points;
85 mec C = enclosing_circle(working_points);
86 }

```

4.3.7. Integer Point

Title: Integer Point

Author: botellot / ICPC BRO

Description: Robust 2D Point/Vector structure using long long to avoid precision issues.

Includes dot product, cross product, sorting comparators.

Complexity: $O(1)$ for all operations.

Limitations:

- Does not support non-integer operations (like division or precise rotation).
- Watch out for overflow in cross/dot products if coordinates are $> 10^9$ (results fit in `__int128` if needed).

Usage:

P a{1, 2}, b{3, 4}, c{5, 6};

```

1 struct P {
2     ll x, y;
3     void read() {
4         cin >> x >> y;
5     }
6     // Operaciones básicas de vectores
7     P operator + (const P& b) const { return P{x + b.x, y + b.y}; }
8     P operator - (const P& b) const { return P{x - b.x, y - b.y}; }
9     void operator += (const P& b) { x += b.x; y += b.y; }
10    void operator -= (const P& b) { x -= b.x; y -= b.y; }
11    // Multiplicación por un escalar
12    P operator * (ll k) const { return P{x * k, y * k}; }
13    // Producto Punto (Dot Product) -> a . b = |a||b|cos(theta)
14    // Utilidad: Proyecciones, ver si un ángulo es agudo (>0), recto (=0) u obtuso (<0)
15    ll dot(const P& b) const { return x * b.x + y * b.y; }
16    // Producto Cruz (Cross Product) -> a x b = |a||b|sin(theta)
17    // Utilidad: Áreas, ver orientaciones y giros
18    ll operator * (const P& b) const { return x * b.y - y * b.x; }
19    // Magnitud al cuadrado (evita usar sqrt y floats)
20    ll norm() const { return x * x + y * y; }
21    // Comparadores (Ordena puntos y puede usar std::sort)
22    // Ordena de izquierda a derecha, y abajo hacia arriba
23    bool operator < (const P& b) const {
24        return make_pair(x,y) < make_pair(b.x, b.y);
25    }
26    bool operator == (const P& b) const {
27        return x == b.x && y == b.y;
28    }
29 };

```

4.3.8. Point in Polygon (Ray Casting)

Title: Point in Polygon (Ray Casting)

Author: botellot / ICPC BRO

Description: Checks if a point P is INSIDE (1), OUTSIDE (0), or on the BOUNDARY (2) of a simple polygon.

Uses a ray to infinity shifted by $X+1$ to completely avoid vertex-intersection edge cases.

Needs vertices of polygon ordered clockwise or anticlockwise.

Complexity: $O(N)$ per query, where N is the number of polygon vertices.

```

1 struct P {
2     ll x, y;
3     void read() { cin >> x >> y; }
4     P operator - (const P& b) const { return {x - b.x, y - b.y}; }
5     __int128 operator * (const P& b) const { return (__int128)x * b.y - (__int128)y
6         * b.x; }
7 };
8 // Determina si el punto c esta sobre el segmento ab
9 bool in_segment(P a, P b, P c) {
10     if ((a - c) * (b - c) != 0) return false; // No son colineales
11     return min(a.x, b.x) <= c.x && c.x <= max(a.x, b.x) &&
12         min(a.y, b.y) <= c.y && c.y <= max(a.y, b.y);
13 }
14 // Interseccion de segmentos
15 bool intersect(P p1, P p2, P p3, P p4) {
16     if ((p4 - p3) * (p2 - p1) == 0) {
17         if ((p2 - p1) * (p3 - p1) != 0) return false;
18         for (int rep = 0; rep < 2; rep++) {
19             if (max(p1.x, p2.x) < min(p3.x, p4.x) || max(p1.y, p2.y) < min(p3.y, p4
20                 .y)) return false;
21             swap(p1, p3); swap(p2, p4);
22         }
23         return true;
24     }
25     for (int rep = 0; rep < 2; rep++) {
26         __int128 s1 = (p2 - p1) * (p3 - p1);
27         __int128 s2 = (p2 - p1) * (p4 - p1);
28         if ((s1 < 0 && s2 < 0) || (s1 > 0 && s2 > 0)) return false;
29         swap(p1, p3); swap(p2, p4);
30     }
31     return true;
32 }
33 // Retorna: 0 = OUTSIDE, 1 = INSIDE, 2 = BOUNDARY
34 int point_in_polygon(const vector<P>& poly, P p) {
35     int n = poly.size();
36     int cnt = 0;
37     // Rayo al "infinito". El truco de p.x + 1 evita que el rayo cruce exactamente
38     // por un vertice entero del poligono, eliminando bugs de doble conteo.
39     P out = {p.x + 1, 2000000007LL};
40     for (int i = 0; i < n; i++) {
41         int j = (i + 1 == n) ? 0 : i + 1;
42         // 1. Si el punto esta sobre el borde de este segmento, terminar de
43         // inmediato
44         if (in_segment(poly[i], poly[j], p)) {
45             return 2;
46         }
47         // 2. Contar intersecciones con el rayo
48         if (intersect(p, out, poly[i], poly[j])) {
49             cnt++;
50         }
51     }
52     // Impar = INSIDE, Par = OUTSIDE
53     return (cnt % 2 == 1) ? 1 : 0;
54 }
55 void solve() {
56     int n, m;
57     if (!(cin >> n >> m)) return;
58     vector<P> polygon(n);
59     for (int i = 0; i < n; i++) polygon[i].read();
60     while (m--) {
61         P p; p.read();
62         int res = point_in_polygon(polygon, p);
63         if (res == 2) cout << "BOUNDARY\n";
64         else if (res == 1) cout << "INSIDE\n";
65         else cout << "OUTSIDE\n";
66     }
67 }

```

```
63 }
64 }
```

4.3.9. Point Location vs Line

Title: Point Location vs Line
Author: botellot / ICPC BRO
Description: Returns orientation of P3 relative to vector P1->P2.
- LEFT (1): Giro Antihorario (Counter-Clockwise). Producto cruz > 0.
- RIGHT (-1): Giro Horario (Clockwise). Producto cruz < 0.
- TOUCH (0): Colineales / Sobre la misma linea. Producto cruz = 0.
Complexity: O(1).

```
1 struct P {
2     ll x, y;
3     P operator - (const P& b) const { return {x - b.x, y - b.y}; }
4     ll operator * (const P& b) const { return x * b.y - y * b.x; }
5 };
6 int point_location(P p1, P p2, P p3) {
7     ll cp = (p2 - p1) * (p3 - p1);
8     if (cp < 0) return -1; // Horario / Derecha
9     if (cp > 0) return 1;  // Antihorario / Izquierda
10    return 0;              // Colineal
11 }
```

4.3.10. Polygon Area (Shoelace Formula)

Title: Polygon Area (Shoelace Formula)
Author: botellot / ICPC BRO
Description: Computes twice the area of a simple polygon (convex or concave).
Returns 2 * Area as long long to maintain absolute integer precision.
Vertices must be given in clockwise or counter-clockwise order.
Complexity: O(N) where N is the number of vertices.
Usage:
vector<P> poly = {{0,0}, {4,0}, {4,3}};
ll double_area = polygon_double_area(poly); // Returns 12
double real_area = double_area / 2.0; // 6.0

```
1 struct P {
2     ll x, y;
3     void read() {cin >> x >> y;}
4     ll operator * (const P& b) const {return x * b.y - y * b.x;}
5 };
6 // Retorna 2 * Area del poligono usando el producto cruz desde el origen
7 ll polygon_double_area(const vector<P>& poly) {
8     int n = (int)poly.size();
9     ll area = 0LL;
10    for (int i = 0; i < n; i++) {
11        int next_idx = (i + 1 == n) ? 0 : i + 1;
12        area += poly[i] * poly[next_idx];
13    }
14    return abs(area);
15 }
16 void solve() {
17     int n; cin >> n;
18     vector<P> polygon(n);
19     for(P& p : polygon) {
20         p.read();
21     }
22     ll area = polygon_double_area(polygon);
23 }
```

4.3.11. Segment Intersection (Anti-Overflow Version)

Title: Segment Intersection (Anti-Overflow Version)
Author: botellot / ICPC BRO
Description: Robustly checks if line segments P1P2 and P3P4 intersect.
Uses __int128 in cross product to strictly avoid overflow with coordinates up to 10⁹.
Handles general intersections, T-junctions, and collinear/overlapping segments.
Complexity: O(1).

```
1 struct P {
2     ll x, y;
3     void read() {
4         cin >> x >> y;
5     }
6     P operator - (const P& b) const {
7         return {x - b.x, y - b.y};
8     }
9     // Retorna __int128 para blindar multiplicaciones de coordenadas de hasta 10^9
10    __int128 operator * (const P& b) const {
11        return (__int128)x * b.y - (__int128)y * b.x;
12    }
13 };
14 bool intersect(P p1, P p2, P p3, P p4) {
15     // Caso 1: Los segmentos son paralelos o colineales
16     if ((p4 - p3) * (p2 - p1) == 0) {
17         // Si no son colineales (paralelos en distintas lineas), no hay
18         // interseccion
19         if ((p2 - p1) * (p3 - p1) != 0) return false;
20         // Si son colineales, revisamos el traslape de sus cajas contenedoras
21         for (int rep = 0; rep < 2; rep++) {
22             if (max(p1.x, p2.x) < min(p3.x, p4.x) || max(p1.y, p2.y) < min(p3.y, p4
23                 .y)) {
24                 return false;
25             }
26             swap(p1, p3); swap(p2, p4);
27         }
28         return true;
29     }
30     // Caso 2: No paralelos (Straddle Test con loop y swap)
31     for (int rep = 0; rep < 2; rep++) {
32         // s1 y s2 guardan los signos de orientacion usando __int128
33         __int128 s1 = (p2 - p1) * (p3 - p1);
34         __int128 s2 = (p2 - p1) * (p4 - p1);
35         // Se descarta si ambos puntos estan estrictamente del mismo lado
36         if ((s1 < 0 && s2 < 0) || (s1 > 0 && s2 > 0)) return false;
37         swap(p1, p3); swap(p2, p4);
38     }
39     return true;
40 }
41 void solve() {
42     P p1, p2, p3, p4;
43     p1.read(); p2.read(); p3.read(); p4.read();
44     if (intersect(p1, p2, p3, p4)) {
45         cout << "YES\n";
46     } else {
47         cout << "NO\n";
48     }
49 }
50 return 0;
```

4.4. Teoria Juegos

4.4.1. XOR Basis (Linear Basis)

Title: XOR Basis (Linear Basis)
Author: ICPC BRO / botellot

Description: Maintains a set of linearly independent vectors under XOR.
 Useful for maximum/minimum XOR subset, counting distinct XOR sums, and membership queries.
 Complexity: $O(\log(\text{MAX_VAL}))$ per insertion. Max size is ~60 for long long.

```

1 struct XorBasis {
2     long long basis[62];
3     int sz = 0; // Guarda cuantos elementos reales tiene la base
4     XorBasis() {
5         for (int i = 0; i < 62; ++i) basis[i] = 0;
6     }
7     // Inserta un numero a la base. Retorna true si aporoto informacion nueva.
8     bool insert(long long x) {
9         for (int i = 61; i >= 0; --i) {
10             if ((x >> i) & 1) { // Si el bit 'i' esta encendido
11                 if (!basis[i]) { // Si la ranura esta vacia, lo insertamos
12                     basis[i] = x;
13                     sz++;
14                     return true;
15                 }
16                 x ^= basis[i]; // Si ya hay alguien, reducimos x y seguimos
17             }
18         }
19         return false; // x se volvio 0, no aporoto nada nuevo
20     }
21     // Retorna el MAXIMO XOR posible que se puede formar con cualquier subconjunto
22     long long getMaxXor() {
23         long long res = 0;
24         for (int i = 61; i >= 0; --i) {
25             // Greedy: si prender el bit 'i' aumenta nuestro resultado, lo tomamos
26             if ((res ^ basis[i]) > res) {
27                 res ^= basis[i];
28             }
29         }
30         return res;
31     }
32     // Verifica si un numero 'x' se puede formar con los elementos de la base
33     bool canForm(long long x) {
34         for (int i = 61; i >= 0; --i) {
35             if ((x >> i) & 1) {
36                 if (!basis[i]) return false; // Falto un vector para apagar este bit
37                 x ^= basis[i];
38             }
39         }
40         return x == 0;
41     }
42 };

```

4.5. Teoria Numeros

4.5.1. BigInt Operations (Unsigned)

Description: Basic BigInt structure for positive integers using base 10^9 .
 * Complexity:
 - Addition/Subtraction: $O(N)$ where N is number of nodes (digits/9).
 - Multi-Int Multiply/Divide/Mod: $O(N)$.
 - BigInt Multiply: $O(N * M)$ quadratic. Fails if total digits > 10^5 .
 - String Conversion: $O(L)$ where L is string length.
 * Limitations:
 - Only supports non-negative integers.
 - less_lnum assumes $a \geq b$ (no negative result support).
 - Division and Modulo are only by 'long long' scalars, not BigInt/BigInt.
 - Multiplication is not FFT-optimized.
 Provides fast I/O and linear operations for small/medium inputs.

Author: botellot / ICPC BRO
 Title: BigInt Operations (Unsigned)
 * Usage:
 BigInt a("123456789123456789"), b(100);
 BigInt c = prod_lnum_int(a, 5);
 ll remainder = mod_lnum_int(a, 1e9+7);

```

1 struct BigInt {
2     static const int BASE = 1e9;
3     vector<int> nodes;
4     BigInt() {}
5     BigInt(long long v) {
6         if (v == 0) nodes.push_back(0);
7         while (v > 0) {
8             nodes.push_back(v % BASE);
9             v /= BASE;
10        }
11    }
12    BigInt(string s) {
13        for (int i = (int)s.length(); i > 0; i -= 9) {
14            if (i < 9)
15                nodes.push_back(stoi(s.substr(0, i)));
16            else
17                nodes.push_back(stoi(s.substr(i - 9, 9)));
18        }
19        trim();
20    }
21    void trim() {
22        while (nodes.size() > 1 && nodes.back() == 0)
23            nodes.pop_back();
24        if (nodes.empty()) nodes.push_back(0);
25    }
26 };
27 // --- OPERACIONES EXTERNAS ---
28 // O(N) - Suma estándar con carry
29 BigInt sum_lnum(const BigInt& a, const BigInt& b) {
30     BigInt res = a;
31     int carry = 0;
32     for (size_t i = 0; i < max(res.nodes.size(), b.nodes.size()) || carry; ++i) {
33         if (i == res.nodes.size()) res.nodes.push_back(0);
34         long long cur = (long long)res.nodes[i] + carry + (i < b.nodes.size() ? b.nodes[i] : 0);
35         res.nodes[i] = cur % BigInt::BASE;
36         carry = cur / BigInt::BASE;
37     }
38     return res;
39 }
40 // O(N) - Resta estándar (Asume a >= b)
41 BigInt less_lnum(const BigInt& a, const BigInt& b) {
42     BigInt res = a;
43     int carry = 0;
44     for (size_t i = 0; i < b.nodes.size() || carry; ++i) {
45         long long cur = res.nodes[i] - carry - (i < b.nodes.size() ? b.nodes[i] : 0);
46         carry = cur < 0;
47         if (carry) cur += BigInt::BASE;
48         res.nodes[i] = (int)cur;
49     }
50     res.trim();
51     return res;
52 }
53 // O(N) - Multiplicación por escalar
54 BigInt prod_lnum_int(const BigInt& a, ll v) {
55     if (v == 0) return BigInt(0);
56     BigInt res = a;
57     ll carry = 0;
58     for (size_t i = 0; i < res.nodes.size() || carry; ++i) {
59         if (i == res.nodes.size()) res.nodes.push_back(0);
60         long long cur = res.nodes[i] * 1LL * v + carry;
61         res.nodes[i] = cur % BigInt::BASE;

```

```

62     carry = cur / BigInt::BASE;
63 }
64 res.trim();
65 return res;
66 }
67 // O(N*M) - Multiplicación BigInt * BigInt
68 BigInt prod_lnum(const BigInt& a, const BigInt& b) {
69     BigInt res;
70     res.nodes.resize(a.nodes.size() + b.nodes.size(), 0);
71     for (size_t i = 0; i < a.nodes.size(); ++i) {
72         ll carry = 0;
73         for (size_t j = 0; j < b.nodes.size() || carry; ++j) {
74             long long cur = res.nodes[i + j] +
75                 a.nodes[i] * 1LL * (j < b.nodes.size() ? b.nodes[j] :
76                     0) + carry;
77             res.nodes[i + j] = cur % BigInt::BASE;
78             carry = cur / BigInt::BASE;
79         }
80     }
81     res.trim();
82     return res;
83 }
84 // O(N) - División por escalar
85 BigInt div_lnum_int(const BigInt& a, ll v) {
86     BigInt res = a;
87     ll carry = 0;
88     for (int i = (int)res.nodes.size() - 1; i >= 0; --i) {
89         long long cur = res.nodes[i] + carry * 1LL * BigInt::BASE;
90         res.nodes[i] = (int)(cur / v);
91         carry = cur % v;
92     }
93     res.trim();
94     return res;
95 }
96 // O(N) - Módulo por escalar
97 ll mod_lnum_int(const BigInt& a, ll v) {
98     ll m = 0;
99     for (int i = (int)a.nodes.size() - 1; i >= 0; --i)
100         m = (a.nodes[i] + m * 1LL * BigInt::BASE) % v;
101     return m;
102 }
103 // O(N) - Print formateado con ceros a la izquierda para la base
104 void print(const BigInt& a) {
105     if (a.nodes.empty()) { printf("0\n"); return; }
106     printf("%d", a.nodes.back());
107     for (int i = (int)a.nodes.size() - 2; i >= 0; --i)
108         printf("%09d", a.nodes[i]);
109     printf("\n");
110 }

```

```

13 vector<char> segmentedSieve(long long L, long long R) {
14     // generate all primes up to sqrt(R)
15     long long lim = sqrt(R);
16     vector<char> mark(lim + 1, false);
17     vector<long long> primes;
18     for (long long i = 2; i <= lim; ++i) {
19         if (!mark[i]) {
20             primes.emplace_back(i);
21             for (long long j = i * i; j <= lim; j += i)
22                 mark[j] = true;
23         }
24     }
25     vector<char> isPrime(R - L + 1, true);
26     for (long long i : primes)
27         for (long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i)
28             isPrime[j - L] = false;
29     if (L == 1)
30         isPrime[0] = false;
31     return isPrime;
32 }

```

4.5.2. Prime Sieve

Title: Prime Sieve

Description: Criba para encontrar primos menores a n y contar primos en segmento [L,R]

Author: bot

```

2 //
3 void criba_n(int n){
4     vector<bool> is_prime(n, true);
5     is_prime[0] = is_prime[1] = false;
6     for (int i = 2; i <= n; i++) {
7         if (is_prime[i] && (long long)i * i <= n) {
8             for (int j = i * i; j <= n; j += i)
9                 is_prime[j] = false;
10         }
11     }
12 }
13 // Cuenta primos en rango con L hasta 1e12 y R-L~1e7

```